

COMP4347 Individual Contribution Reflection

William Tran

1. Subsystem Responsibility and Implementation

My primary responsibility within the project was the administrator subsystem and frontend-backend integration. This included implementing the admin panel UI, protected admin routes, CRUD functionality for quiz questions, bulk question import, frontend validation, backend validation, and authentication-aware navigation. The subsystem was implemented using the MERN stack. The frontend used React and React Router, while the backend used Express.js, MongoDB, and Mongoose. The admin panel supported:

1. Creating, editing, deleting, and toggling questions
2. Bulk importing questions using JSON
3. Admin-only access control
4. Validation for question structure and inputs
5. Modal-based edit/delete workflows

A key part of my implementation was ensuring frontend and backend consistency. I implemented validation checks for question text length, exactly four options, non-empty options, and correct answer bounds. I also contributed significantly to the authentication subsystem, including login and registration functionality using JWT-based authentication. This involved implementing frontend login/register flows, storing authentication state using localStorage, protected frontend routing, backend authentication integration, and role-based navigation rendering. The authentication system ensured that:

1. Unauthenticated users could not access protected quiz/admin routes
2. Admin-only routes were restricted by role
3. Login state persisted across page refreshes
4. Navigation updated dynamically after login/logout actions

I also helped resolve frontend authentication synchronisation issues where navigation state initially failed to update until manual refreshes occurred.

2. Major Technical Challenge and Resolution

One major challenge involved frontend-backend integration for the admin question toggle functionality. Although the backend PATCH route worked correctly through terminal requests, browser requests failed due to CORS preflight validation. The browser produced the error: "Method PATCH is not allowed by Access-Control-Allow-Methods" I resolved this by updating the Express CORS configuration to explicitly allow PATCH requests.

Another challenge involved React authentication state updates. After login, navigation links did not update until the page was manually refreshed because React state was not synchronised with localStorage changes. I resolved this by introducing application-level authentication state handling in App.js, allowing the UI to update immediately after login/logout events.

I also resolved merge conflicts during integration when combining leaderboard functionality with protected routing and admin navigation.

3. Mermaid Sequence Diagram

Figures 1 and 2 in the appendix illustrate the authentication/login-sign-up workflow and the administrator question management workflow, respectively.

4. Git Commit History Analysis

Throughout development, I made consistent commits focused on integration, validation, frontend/backend synchronisation, and admin functionality

Commit Hash	Contribution
cb5ee59	Initial project structure setup, including frontend/backend folder separation, npm development environments, dependency setup, and base MERN architecture configuration
30b250d	Implemented backend authentication system with MongoDB user model, password hashing using bcrypt, JWT login/register endpoints, and authentication middleware integration
31ce626	Developed frontend authentication functionality, including login/register pages, API integration, JWT localStorage handling, protected route support, and authentication-aware navigation
0e46517	Added backend administrator functionality, including CRUD routes for quiz questions, question validation logic, active/inactive toggling, and protected admin endpoints
596069e	Implemented frontend admin panel functionality, including create/edit/delete question workflows, bulk import support, API integration, and frontend validation handling
b7fa39e	Refactored database connection logic into a dedicated db.js configuration module to improve maintainability and backend structure organisation
dd653c5	Fixed bulk import validation and improved frontend/backend error messaging consistency for invalid question formats and incorrect input handling
7c71257	Improved leaderboard styling and frontend presentation, including ranking visuals, alignment fixes, colour highlighting for top users, and UI polish
ce35fc7	Added styling updates and usability improvements for the admin panel, including modal popups, custom dropdown interactions, animated buttons, spacing fixes, and improved layout consistency
9c474bf	Merged main branch into leaderboard feature branch and manually resolved frontend/backend integration conflicts involving routing, authentication, and navigation behaviour
9491bc5	Merged main branch into leaderboard-polish branch and resolved merge conflicts while preserving leaderboard styling updates and protected route functionality
8426067	Reviewed and merged pull request integrating frontend admin functionality, including validating routing compatibility, testing admin workflows, and resolving integration issues with existing frontend navigation and authentication behaviour

User stories, assigned tasks, and implementation progress related to my contributions were tracked through the project Trello board:
<https://trello.com/invite/b/69fade440f06081fee26a24b/ATTIfbc0dfcd82a195a692078f5582e5ea744CB6944/comp4347-trello>

5. Reflection on Design Decisions and Group Variation

A major design decision was separating administrator functionality from standard user quiz functionality using role-based routing and backend authorisation checks. This improved security, maintainability, and user experience. Another important decision was implementing question activation/deactivation rather than relying only on permanent deletion. This allowed administrators to temporarily disable questions without losing stored data.

I also prioritised robustness by implementing validation on both the frontend and backend. Bulk imports, question creation, and editing workflows were all validated before database insertion. The modal-based editing workflow was another deliberate UX improvement. Instead of reusing the creation form, editing occurred inside pop-up overlays, which created a clearer separation between creating and editing questions.

Overall, my contributions focused on integration quality, robustness, frontend/backend consistency, and improving the usability of the administrator subsystem.

6. Appendix

Figure 1: Admin Mermaid Diagram

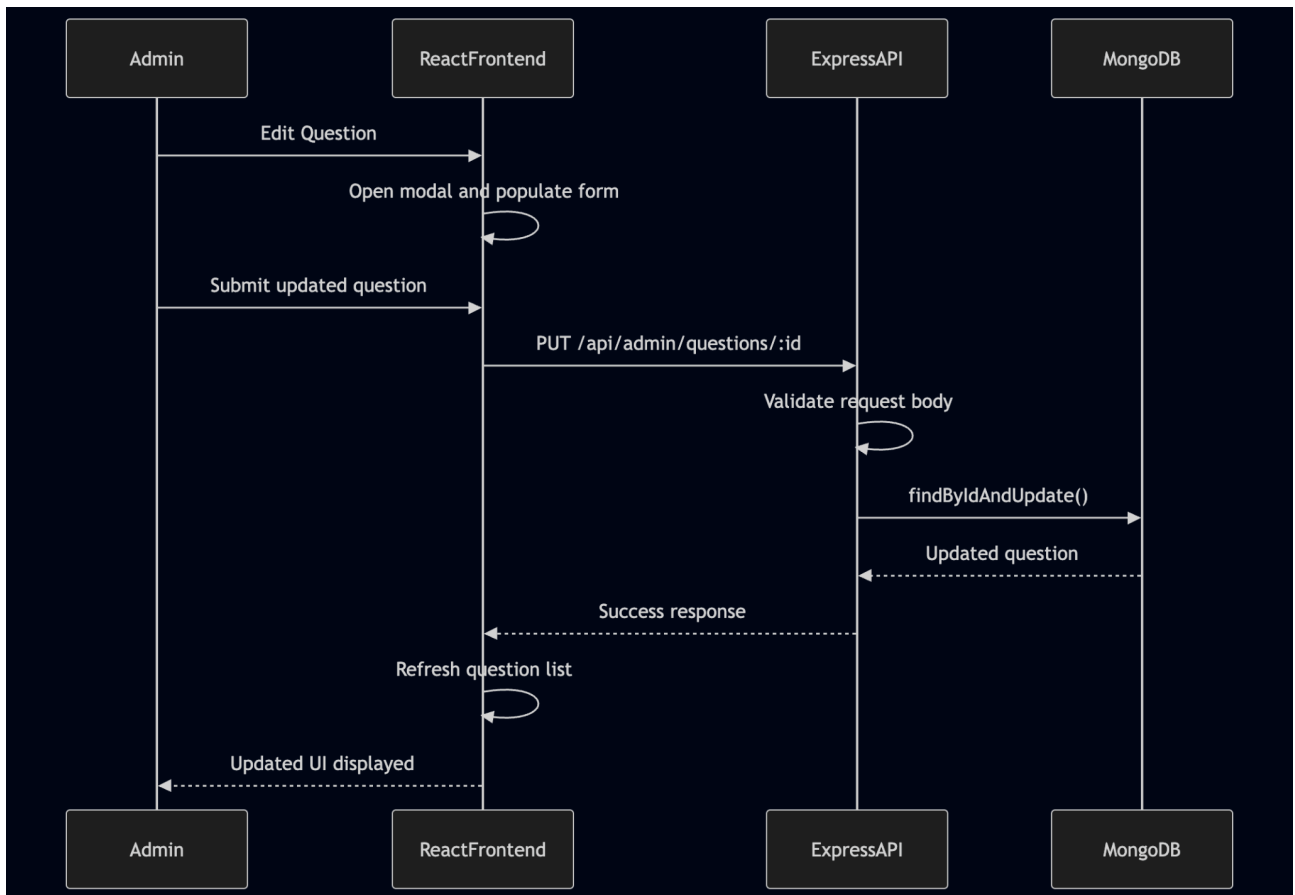


Figure 2: Authentication (Login/Signup) Mermaid Diagram

